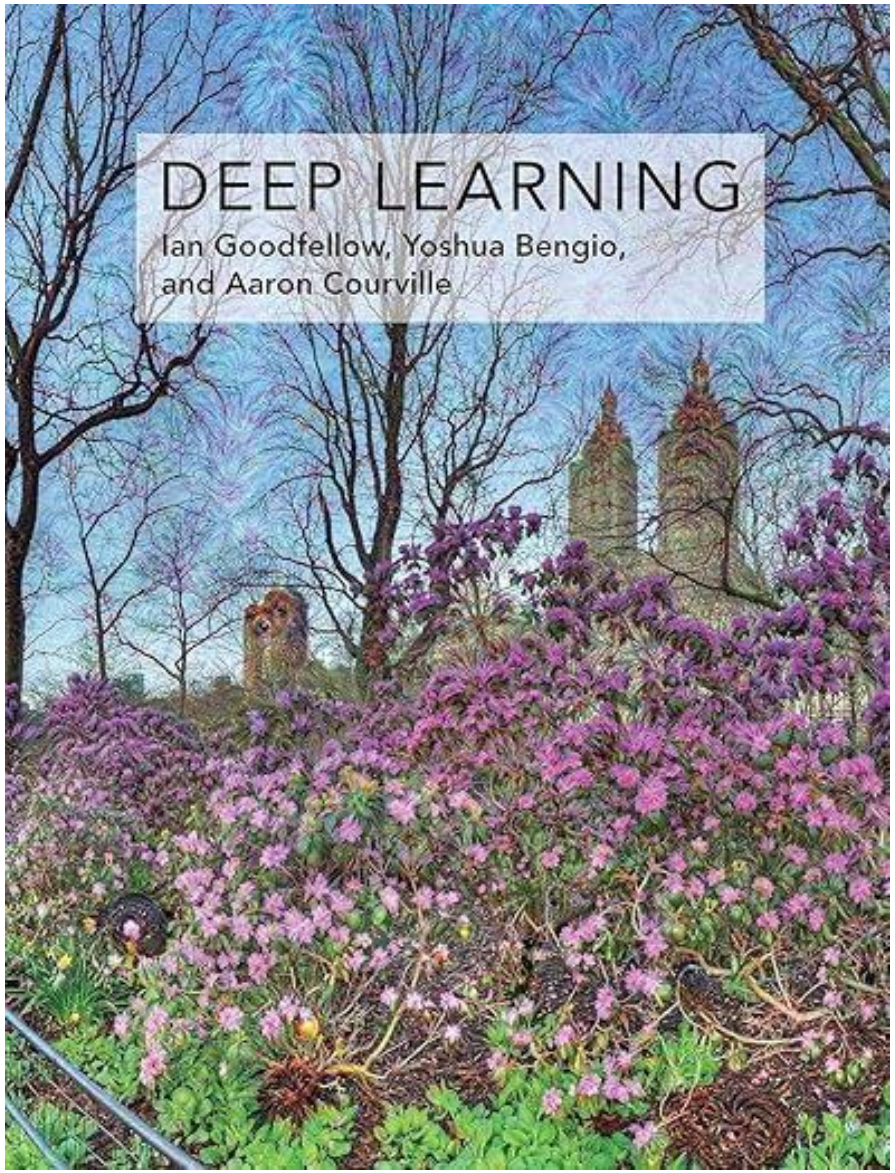
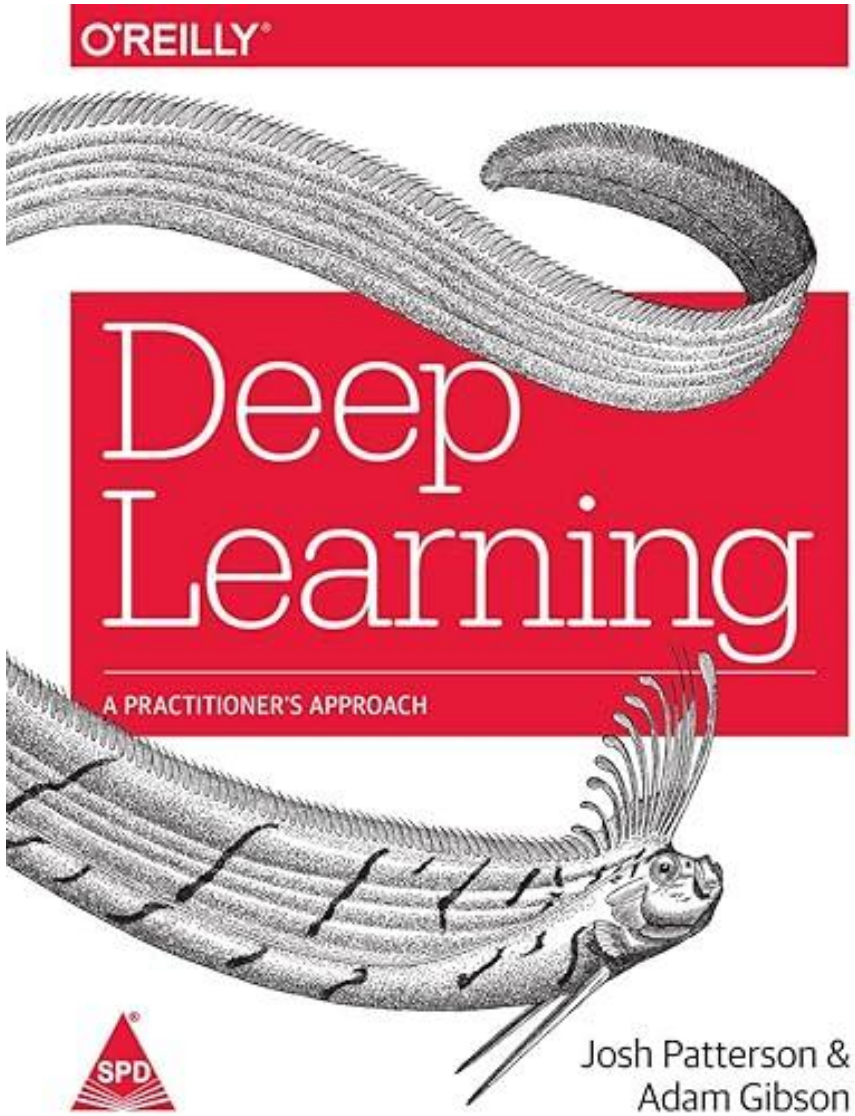
Deep Learning (23AID211)

Lecture-1: Introduction Neural Networks and Deep Neural Networks

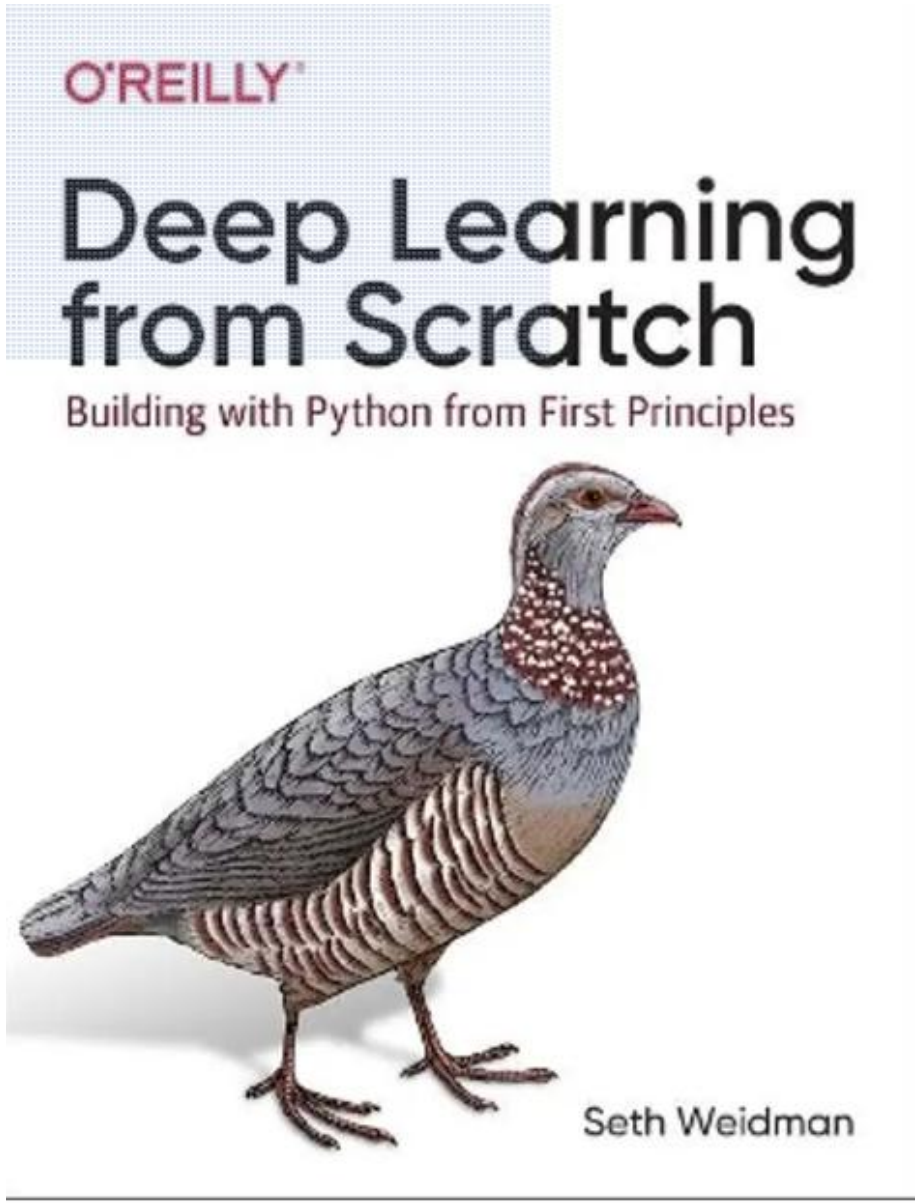
By
Dr. Vivek Patel
(B.Tech, M.Tech, Ph.D)
School of Artificial Intelligence, Amrita Vishwa Vidyapeetham
Faridabad Campus, Delhi NCR, India

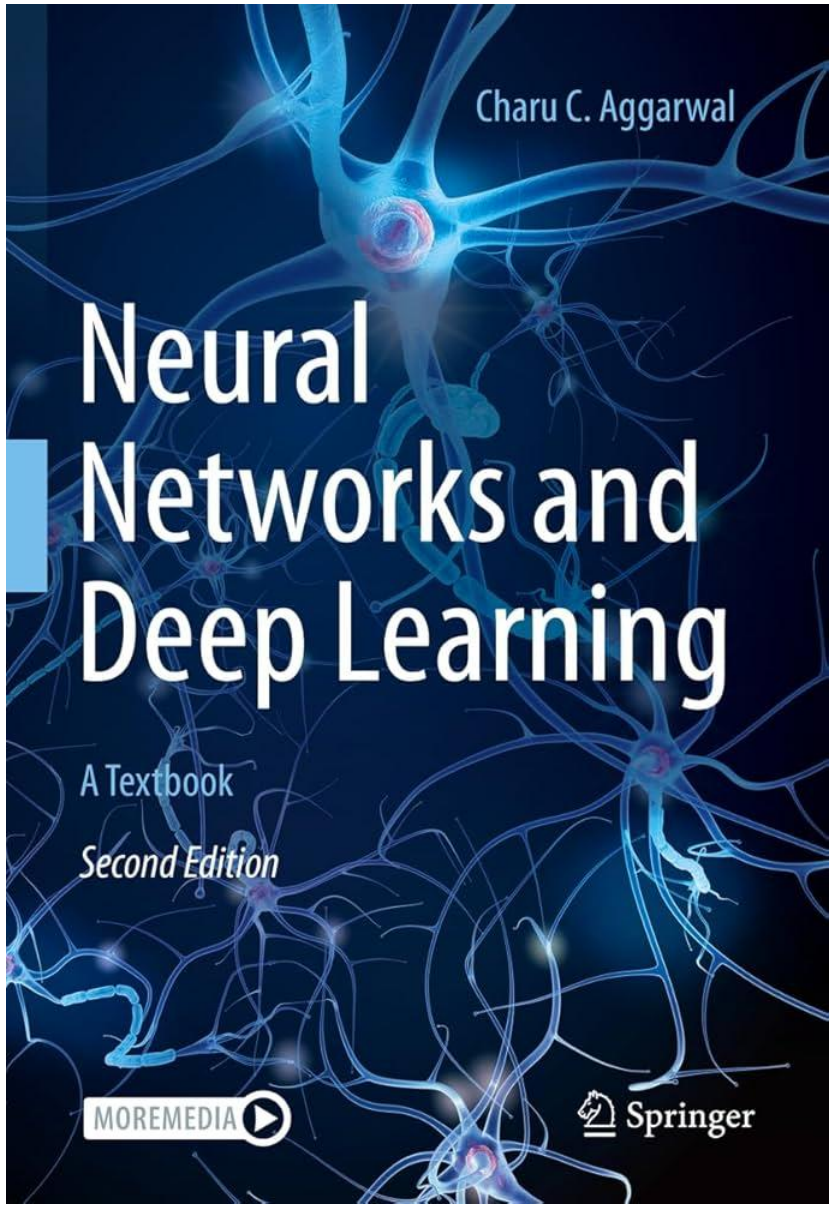
Books


DEEP LEARNING
Ian Goodfellow, Yoshua Bengio,
and Aaron Courville

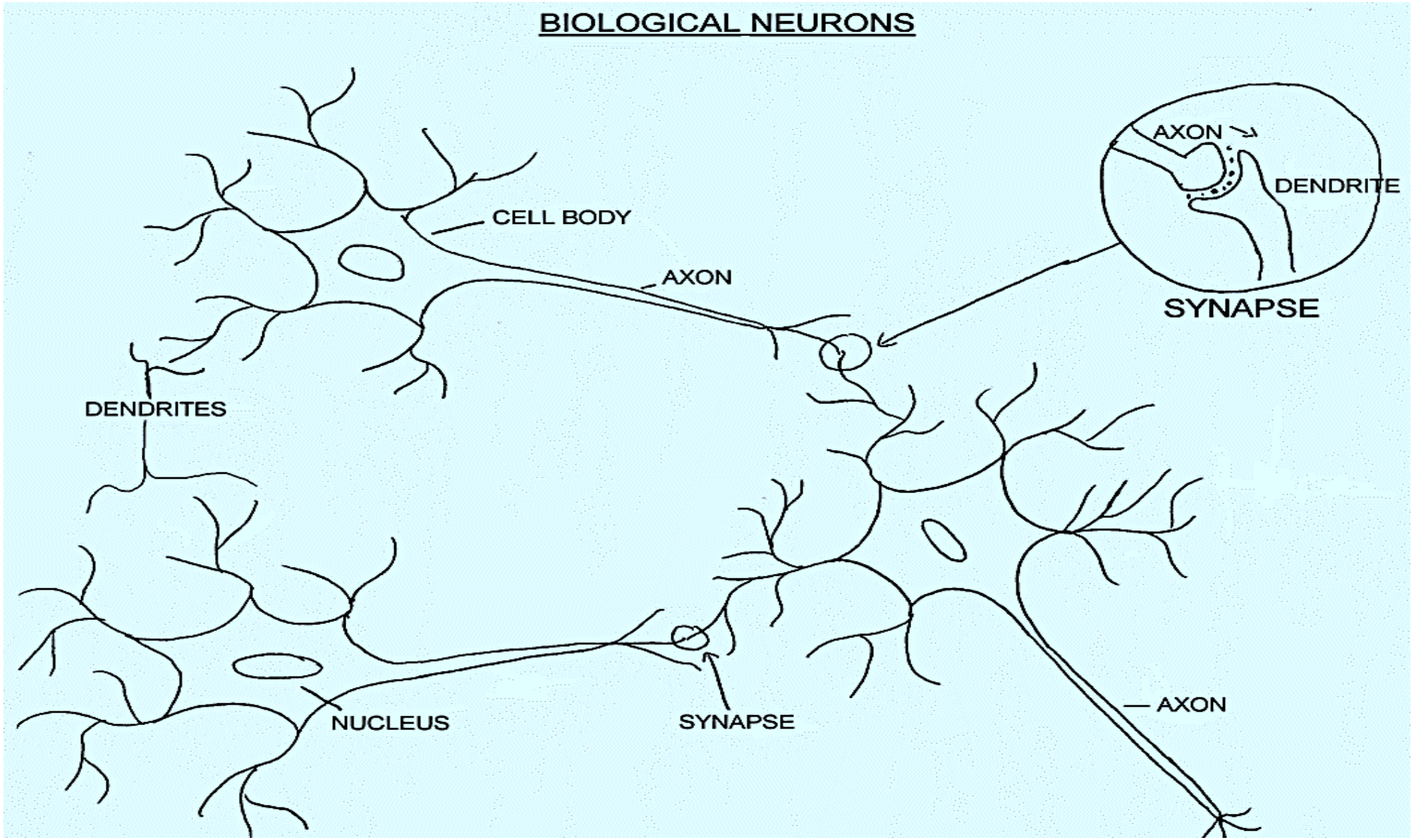

O'REILLY
Deep
Learning
A PRACTITIONER'S APPROACH
SPD
Josh Patterson &
Adam Gibson

Books

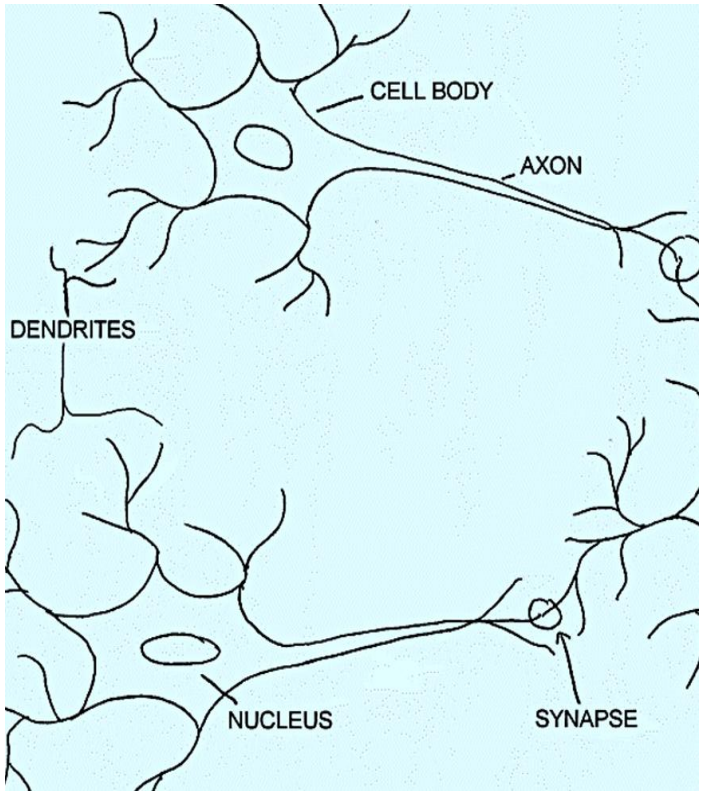

O'REILLY
Deep Learning from Scratch
Building with Python from First Principles
Seth Weidman

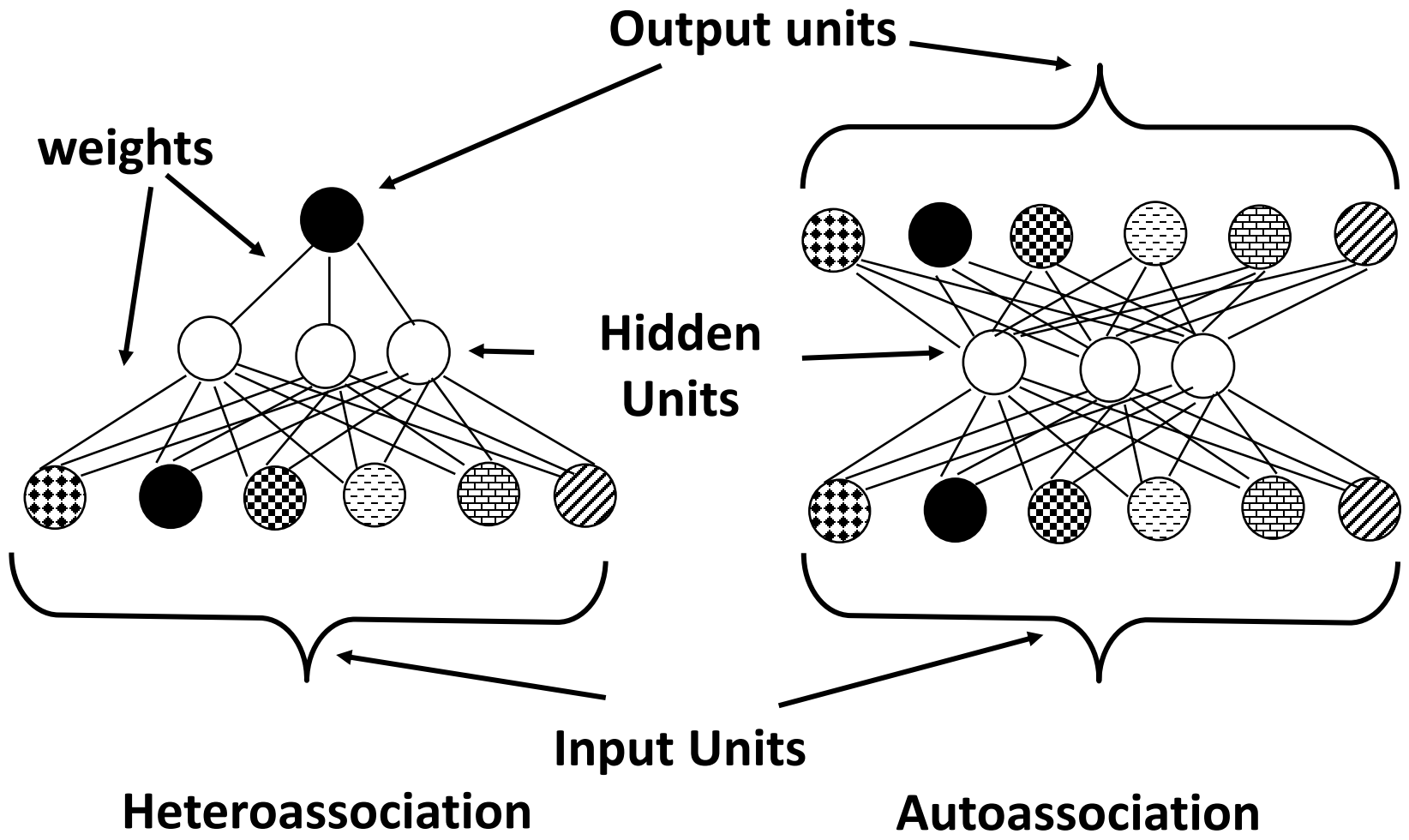

Charu C. Aggarwal
Neural Networks and Deep Learning
A Textbook
Second Edition
MOREMEDIA
Springer

Neural Networks



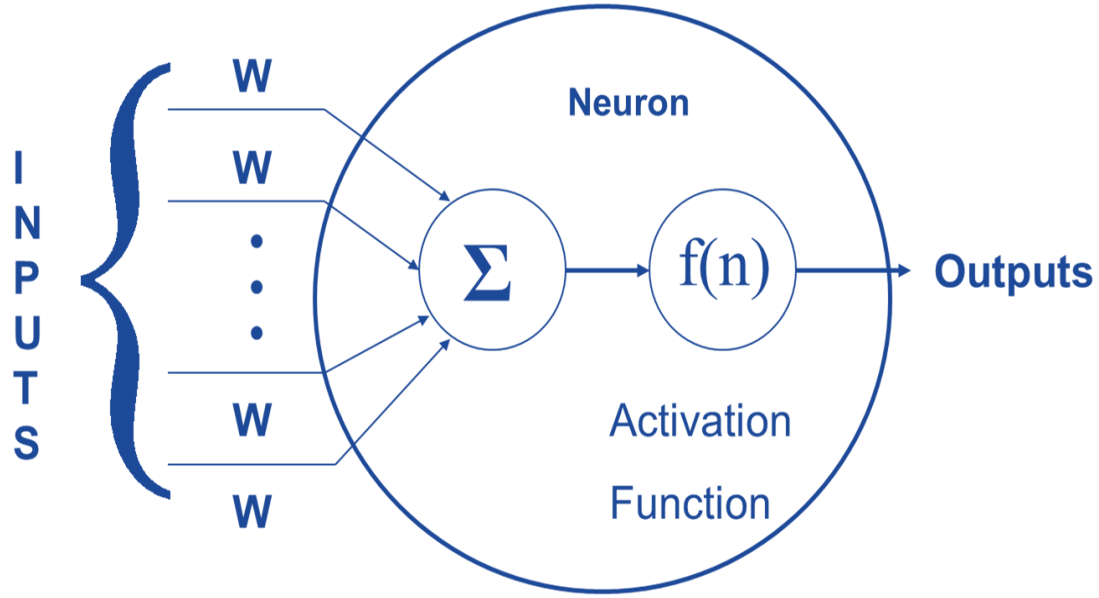
Neural Networks





Neural Networks

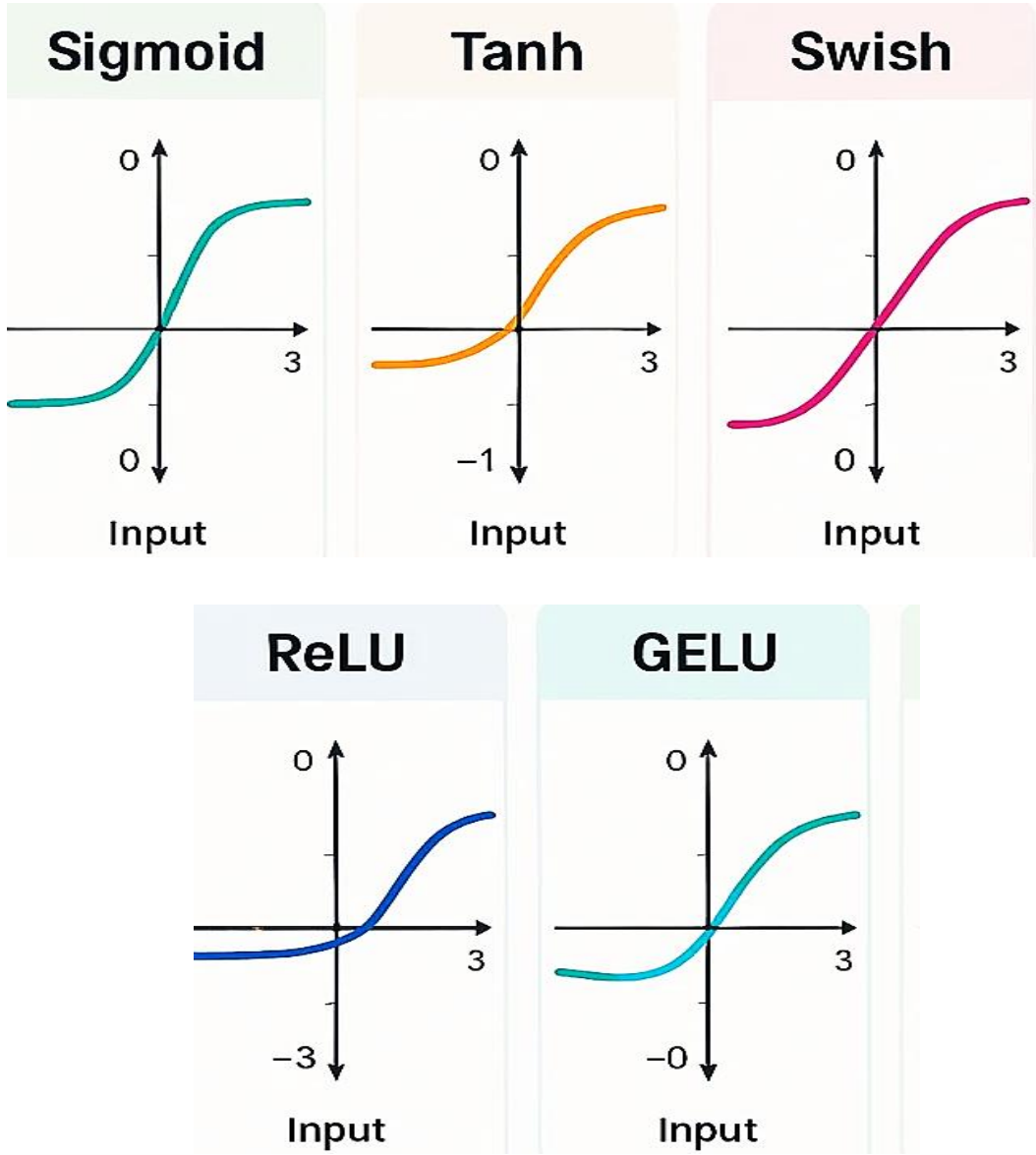
- An activation function in a neural network is a mathematical function applied to the output of a neuron.
- It introduces non-linearity, enabling the model to learn and represent complex data patterns.
- Without it, even a deep neural network would behave like a **simple linear regression model**.



- Activation functions decide whether a neuron should be activated based on the weighted sum of inputs and a bias term.

Activation functions in Neural Networks

Sigmoid	$\frac{1}{1+e^{-x}}$
Tanh	$\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
ReLU	$\max(0, x)$
Leaky ReLU	$\max(0.01x, x)$
GELU	$x * \frac{1}{2}[1+\mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)]$ where $\mathrm{erf}(x) = \frac{2}{\sqrt{\pi}}\int_0^x e^{-t^2}\,dt$



Sigmoid Activation Function

The derivative of the sigmoid function, $\sigma(x) = \frac{1}{1+e^{-x}}$, is $\sigma'(x) = \sigma(x)(1-\sigma(x))$.

This compact form is derived using the chain rule and is crucial for backpropagation in neural networks. It is differentiable everywhere, has a maximum value of 0.25 at $x = 0$, and helps adjust model weights.

Key Details on the Derivative of the Sigmoid Function

- **Formula:** The derivative is expressed in terms of the function itself:
  $\frac{d}{dx}\sigma(x) = \sigma(x) \cdot (1-\sigma(x))$.
- **Derivation:**

  1. Start with the sigmoid function: $\sigma(x) = (1+e^{-x})^{-1}$.

  2. Apply the chain rule: $\frac{d}{dx}(1+e^{-x})^{-1} = -1(1+e^{-x})^{-2} \cdot (-e^{-x})$.

Conti…

3. Simplify to: $\frac{e^{-x}}{(1+e^{-x})^2}$.

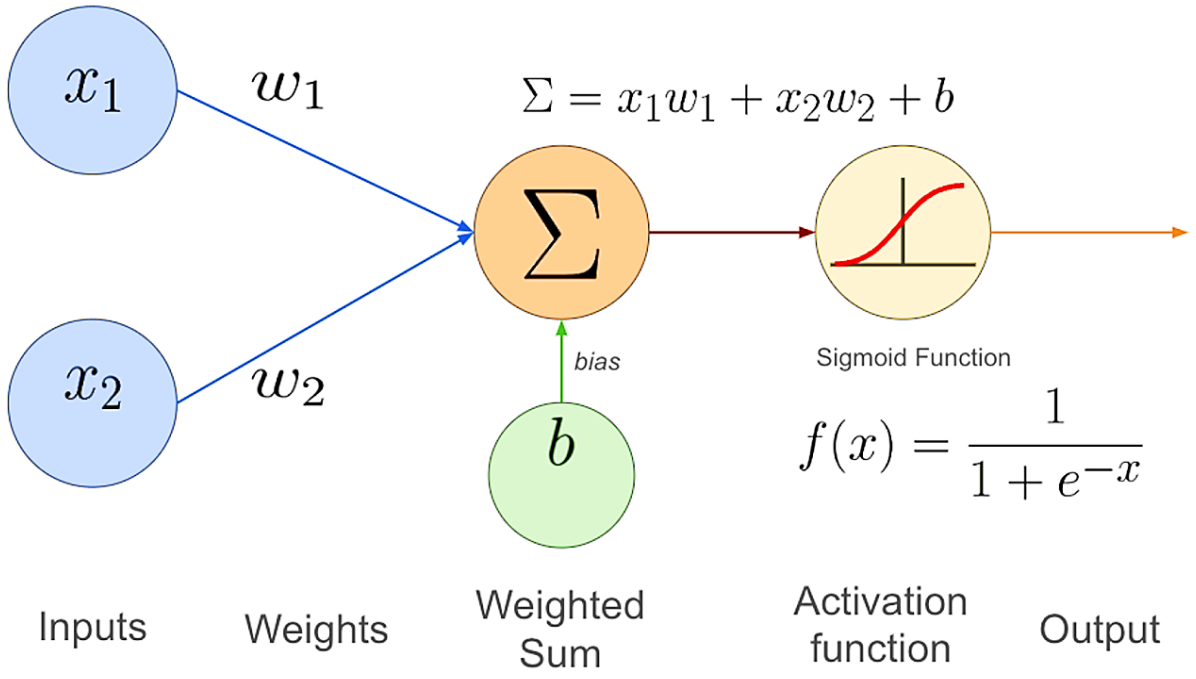
4. Rearrange to express in terms of $\sigma(x)$:

$$\left(\frac{1}{1+e^{-x}}\right)\left(\frac{e^{-x}}{1+e^{-x}}\right) = \sigma(x)(1-\sigma(x)).$$

- **Properties:**
  - **Maximum Value:** The maximum value of the derivative is 0.25, which occurs at $x = 0$.
  - **Range:** The derivative is always positive and maps to the range $(0, 0.25]$.
  - **Usage:** Used in neural networks to compute gradients during backpropagation.
- **Limitations:** The derivative can lead to vanishing gradient problems because its maximum value is relatively small (0.25), which can slow down training in deep networks.

Perceptron Model

- Weights determine how strongly each input contributes to the prediction. A larger weight means the corresponding input has a higher impact.
- Weights are learned during training, adjusting based on errors.
- They act like importance scores for each feature.



The bias is a constant value added to the weighted sum to shift the decision boundary.

- It allows the perceptron to classify correctly even when all input features are zero.
- Bias ensures the model is not forced to pass the decision boundary through the origin.

Perceptron Model

This is the combined effect of all inputs and their weights:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

- Represents the activation strength before passing through the activation function.
- If z is high or low enough, it determines the final class.

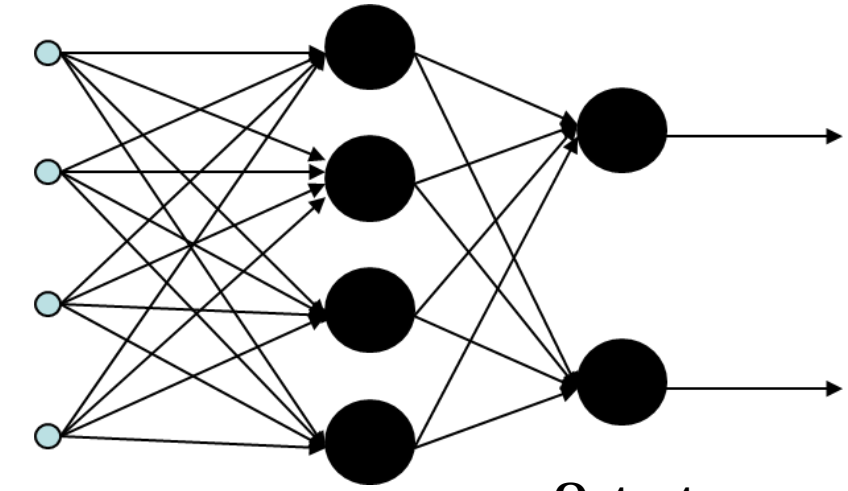
The activation function converts the numerical input into a binary output:

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

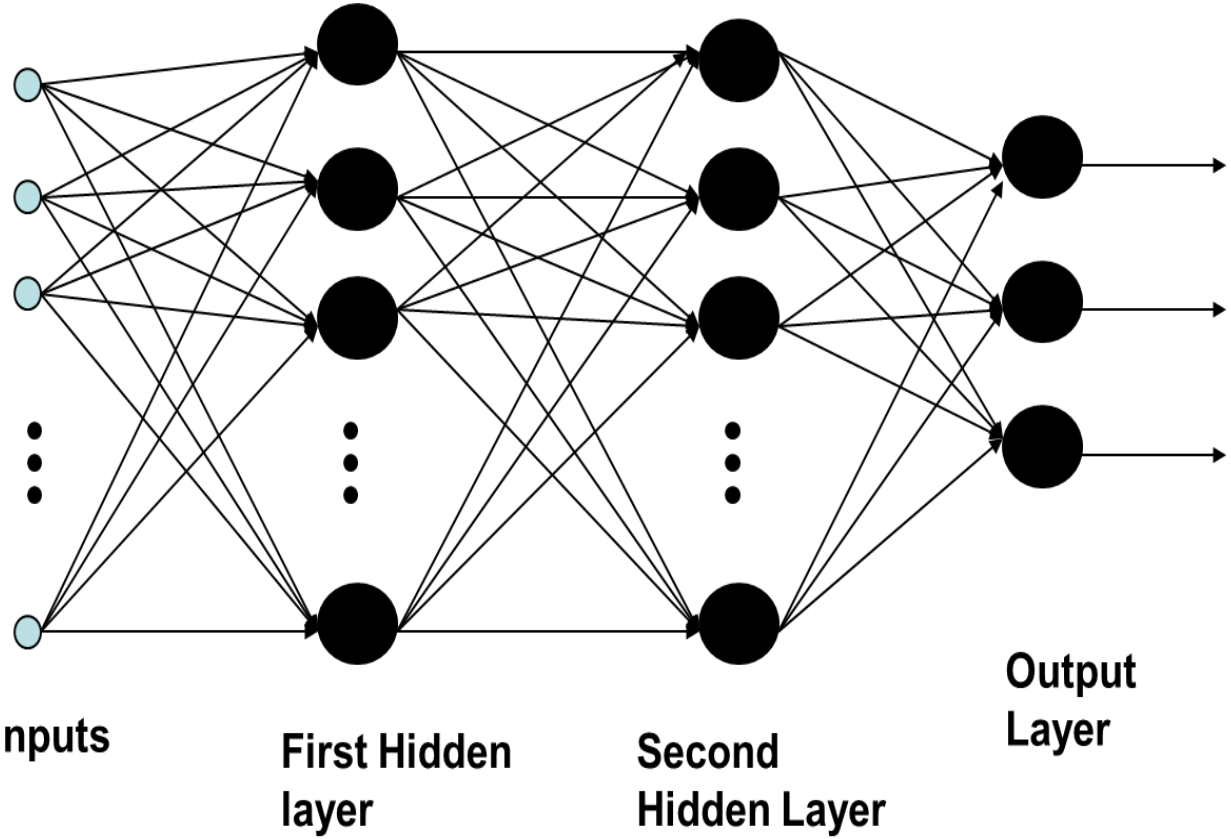
- It introduces non-linearity in the decision-making, although the decision boundary remains linear.
- Output is always 0 or 1 making perceptrons suitable for binary classification.

Simple and Deep Neural Network

Simple Neural Network



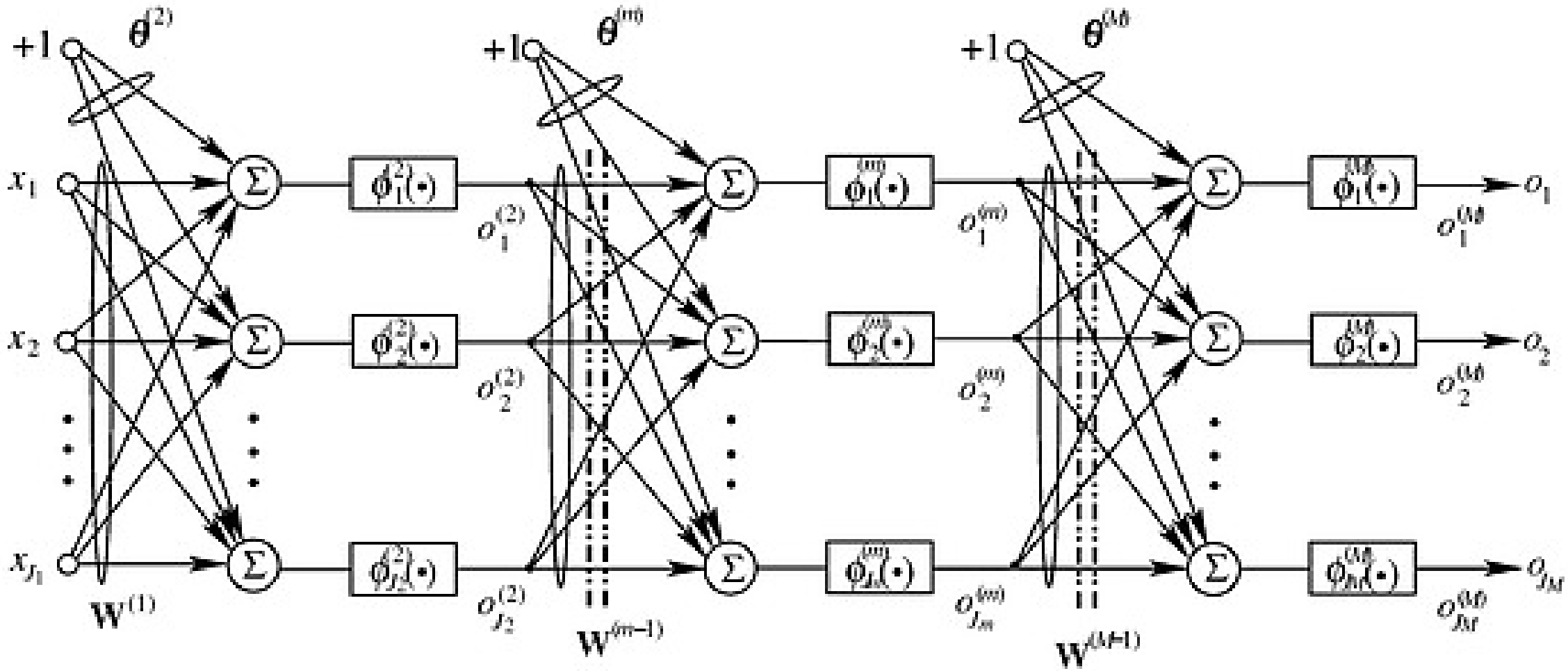
Deep Neural Network



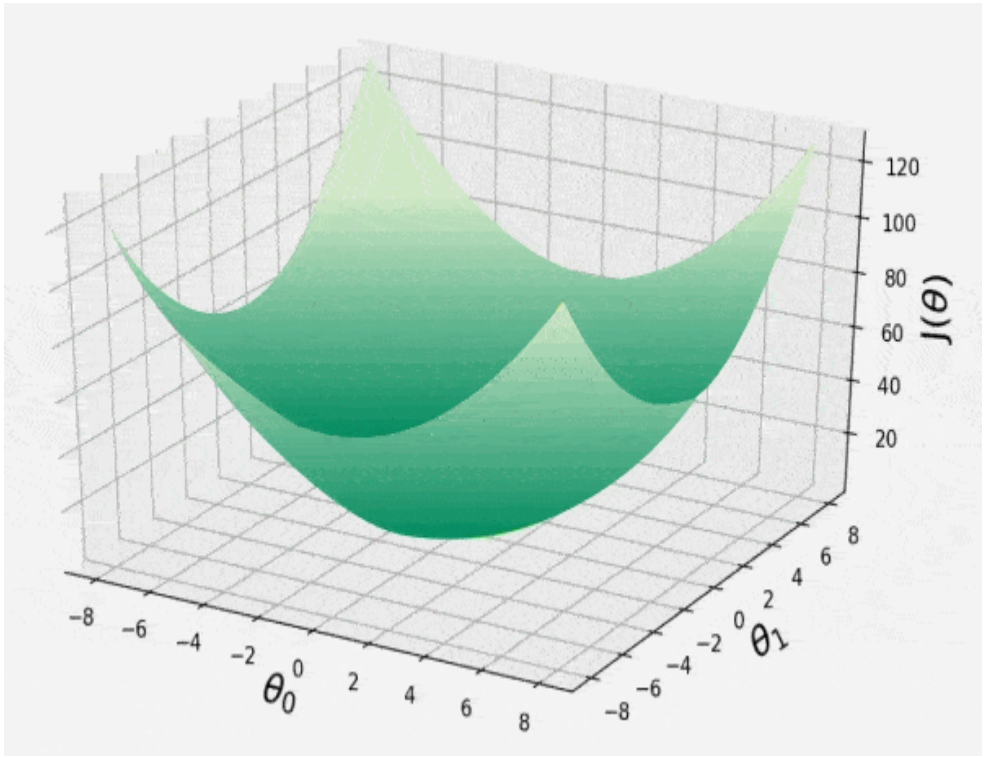
Perceptron vs. Multi-Layer Perceptron (MLP)

Aspect	Perceptron	Multi-Layer Perceptron (MLP)
Model Depth	Single layer with no hidden neurons.	Multiple layers with one or more hidden layers.
Type of Patterns Learned	Learns only linear relationships; straight-line separation.	Learns complex, non-linear patterns and curved boundaries.
Problem-Solving Ability	Cannot solve XOR or non-linearly separable problems.	Easily solves XOR and other complex classification tasks.
Activation Functions	Uses a simple step function (hard 0/1 output).	Uses advanced activations like ReLU, Sigmoid, Tanh for richer learning.
Learning Method	Trained with a simple perceptron update rule.	Trained using backpropagation and gradient descent.
Real-World Use	Limited to simple demonstrations and basic classification.	Used in real-world AI systems like vision, NLP and deep learning applications.

Multi-Layer Perceptron (MLP)



Gradient Descent



- Gradient Descent is an iterative optimization algorithm used to minimize a cost function by adjusting model parameters in the direction of the steepest descent of the function's gradient.
- In simple terms, it finds the optimal values of weights and biases by gradually reducing the error between predicted and actual outputs.

Artificial Neural Network Learning Rules

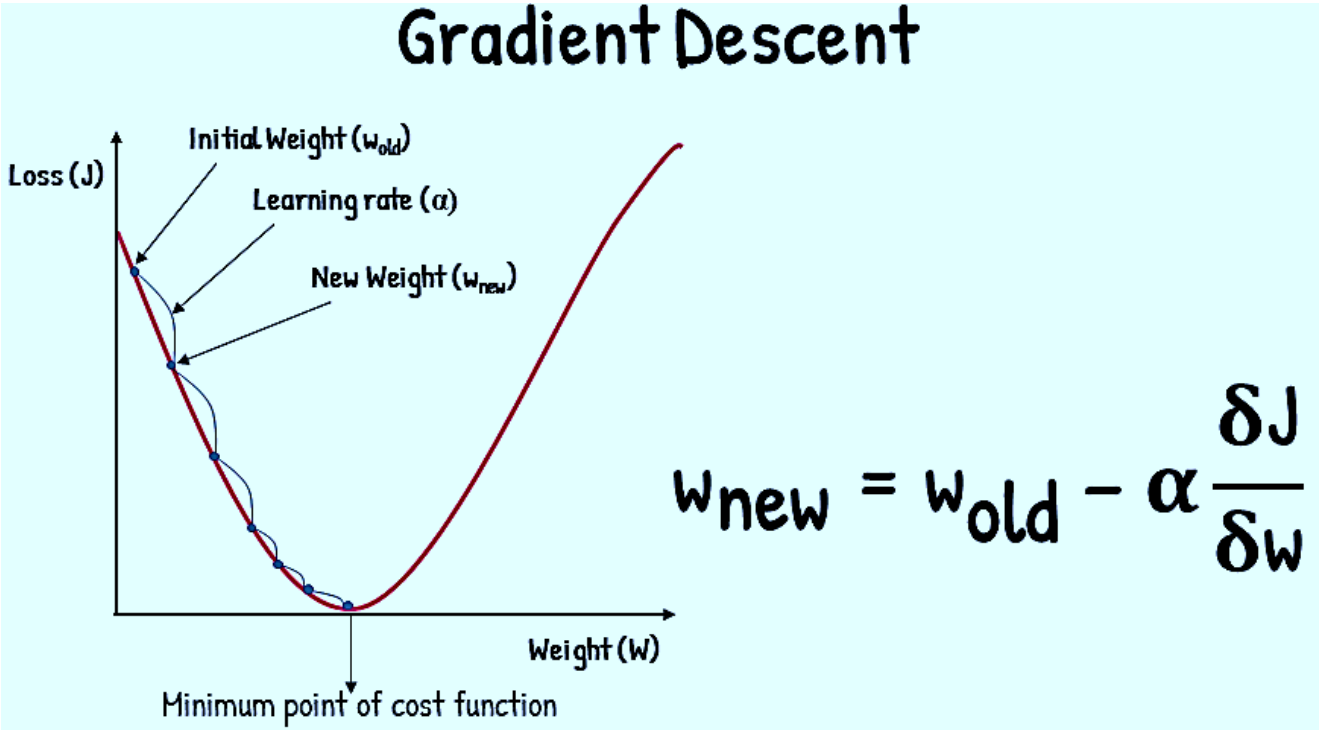
Hebbian Learning

Developed by Donald Hebb in 1949 to describe biological neuron firing. In the mid-1950s it was also applied to computer simulations of neural networks.

Hebbian Learning	Perceptron Learning	Widrow-Hoff Learning (Delta Learning)	Gradient Descent Learning
$\Delta w_i = \boldsymbol{\eta} \,.\, x_i \,.\, \hat{y}_0$ As there is no existence of actual or ground truth or target output in the weight gradient so, it is used for unsupervised learning of the neural network.	$\Delta w_i = \boldsymbol{\eta} \,.\, (y_0 - \hat{y}_0) \,.\, x_i$ This learning rule is rule is used for supervised learning of the neural network because of ground truth.	$\Delta w_i = \boldsymbol{\eta} \,.\, (y_0 - \hat{y}_0) \,.\, f' \,.\, x_i$ f' is the derivative of the activation function. This learning rule is rule is used for supervised learning of the neural network because of ground truth.	$\Delta w_i = \boldsymbol{\eta} \,.\, \frac{\partial E}{\partial w}$ This learning rule is rule is used for supervised learning of the neural network because of ground truth.

Gradient Descent

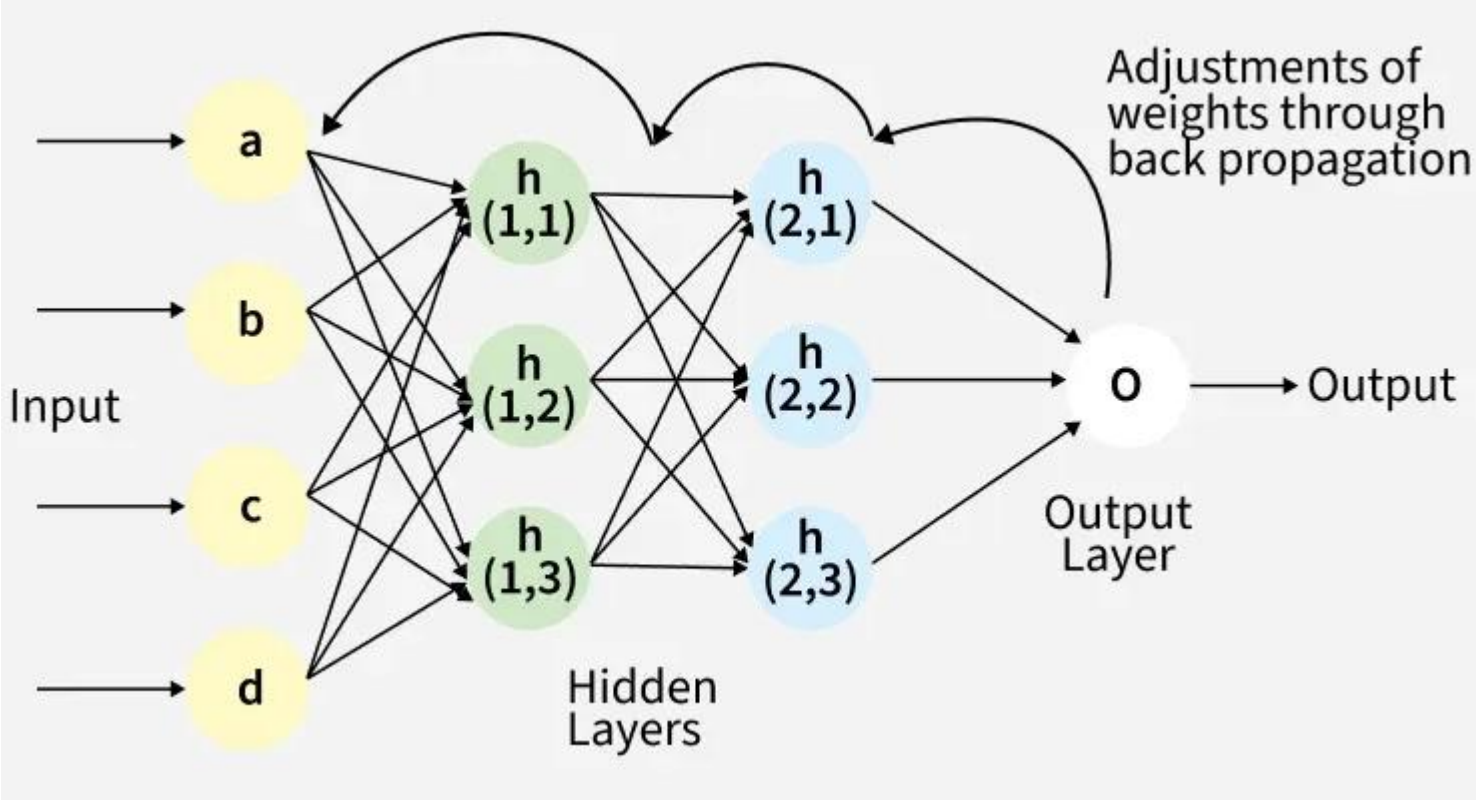
- Backpropagation, short for Backward Propagation of Errors, is a key algorithm used to train neural networks by minimizing the difference between predicted and actual outputs.



- It works by propagating errors backward through the network, using the chain rule of calculus to compute gradients and then iteratively updating the weights and biases.

Back Propagation

- Backpropagation, short for Backward Propagation of Errors, is a key algorithm used to train neural networks by minimizing the difference between predicted and actual outputs.



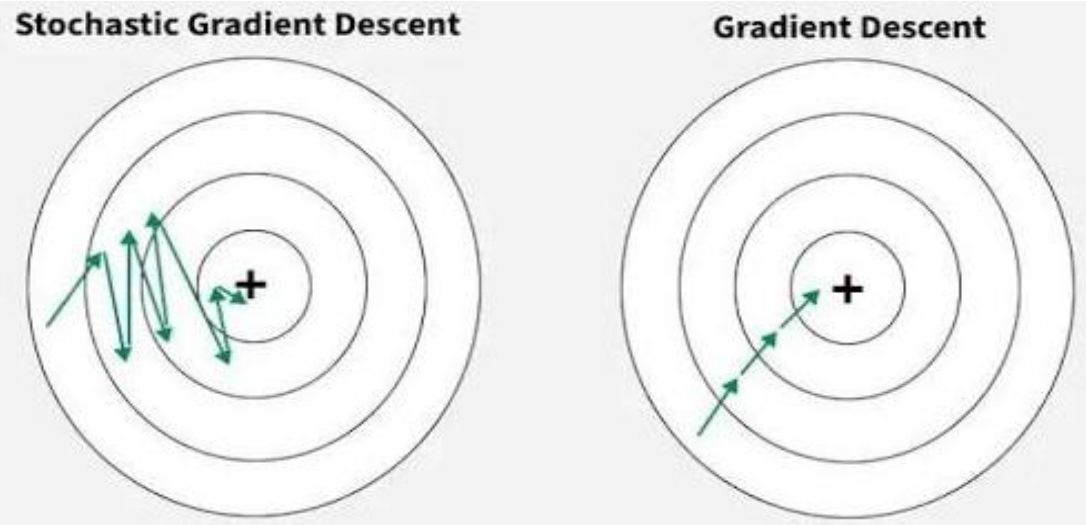
- It works by propagating errors backward through the network, using the chain rule of calculus to compute gradients and then iteratively updating the weights and biases.

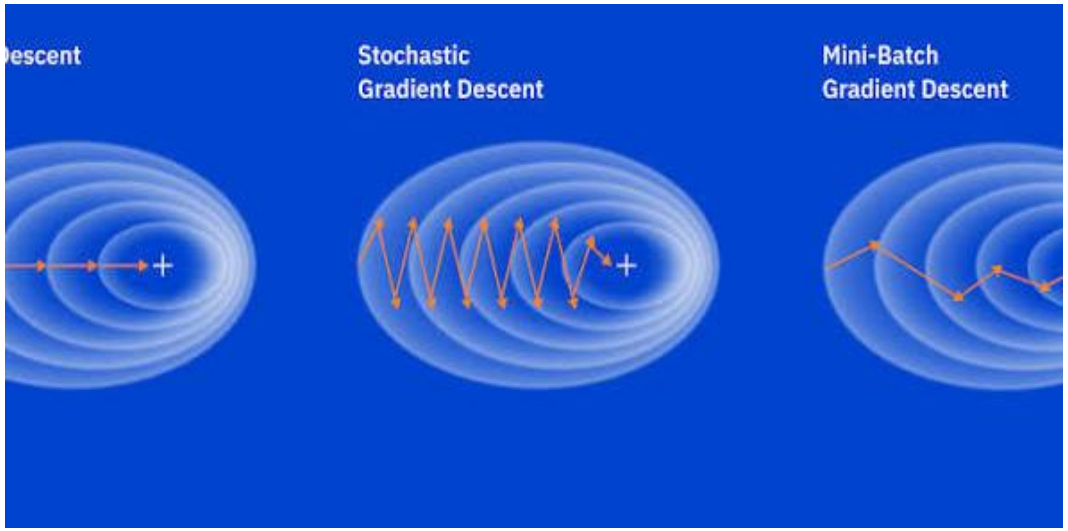
Challenges During Training

While Back Propagation is useful it does face some challenges:

- **Vanishing Gradient Problem**: In deep networks the gradients can become very small during Back Propagation making it difficult for the network to learn. This is common when using activation functions like sigmoid or tanh.
- **Exploding Gradients**: The gradients can also become excessively large causing the network to diverge during training.
- **Overfitting:** If the network is too complex it might memorize the training data instead of learning general patterns.

Gradient Descent (GD) & Stochastic Gradient Descent (SGD)





- Gradient Descent (GD) updates neural network weights by calculating the gradient of the loss function across the entire dataset, offering stable but slow convergence.
- Stochastic Gradient Descent (SGD) updates weights using only one or a mini-batch of samples, providing faster, noisier updates that help escape local minima and speed up training on large datasets.

Conti…

- **Gradient Descent (Batch):** Updates parameters θ using the gradient of the cost function J averaged over all n training examples:
  $\theta = \theta - \eta \cdot \nabla_\theta J(\theta)$
  where η is the learning rate.

- **Stochastic Gradient Descent (SGD):** Updates parameters θ using the gradient of the cost function J based on a single training example $(x^{(i)}, y^{(i)})$ or a small mini-batch:
  $\theta = \theta - \eta \cdot \nabla_\theta J(\theta; x^{(i)}, y^{(i)})$

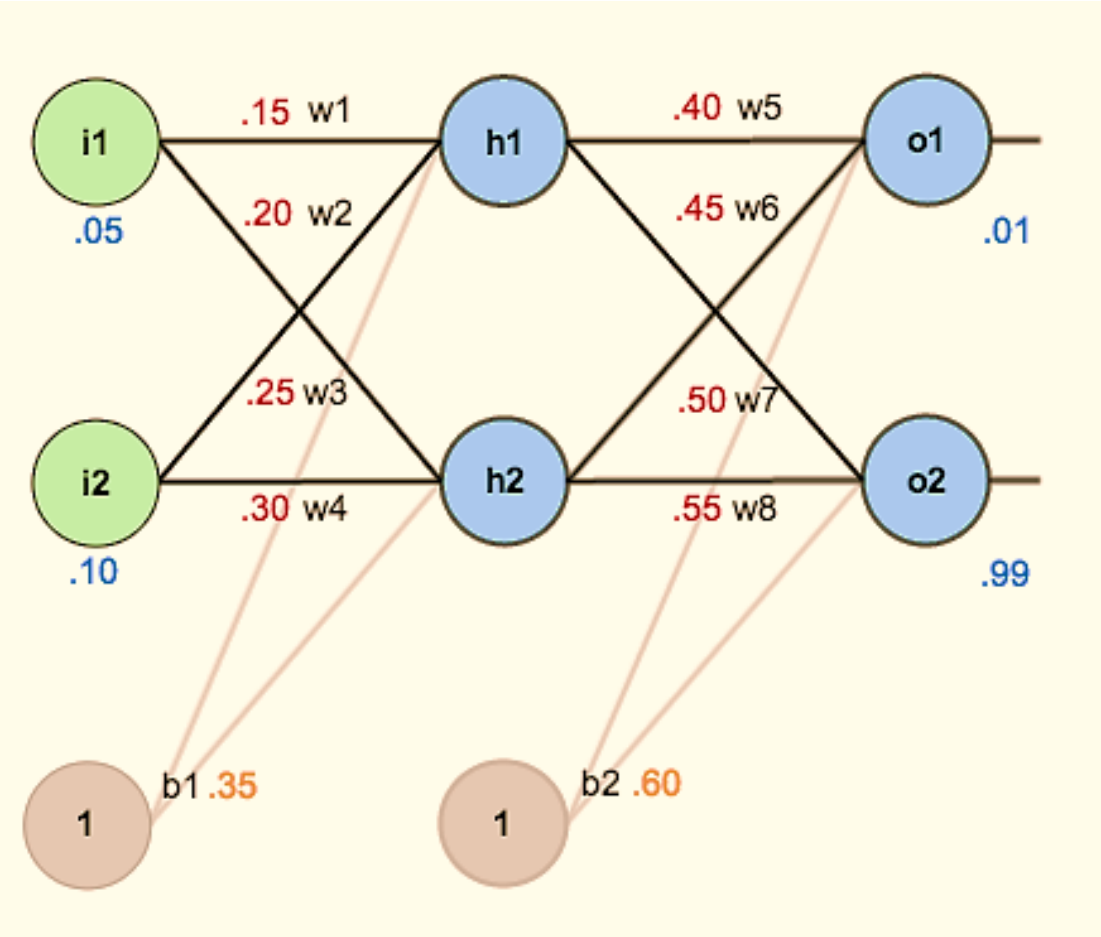
Conti…

Key Differences

- **Data Usage:** GD uses the entire dataset, while SGD uses one or few samples.
- **Convergence & Stability:** GD moves directly toward the minimum, while SGD fluctuates due to the noisy estimates of the gradient, which helps escape local minima and saddle points.
- **Computational Speed:** SGD is faster for large datasets because it does not require loading the entire dataset into memory.
- **Mini-batch GD:** A compromise commonly used in neural networks, blending the efficiency of SGD with the stability of GD.

Numerical Example:

- The goal of backpropagation is to optimize the weights so that the neural network can learn how to correctly map arbitrary inputs to outputs.
- For the rest of this tutorial we're going to work with a single training set: given inputs 0.05 and 0.10, we want the neural network to output 0.01 and 0.99.



The Forward Pass

- To begin, lets see what the neural network currently predicts given the weights and biases above and inputs of 0.05 and 0.10. To do this we'll feed those inputs forward though the network.
- We figure out the *total net input* to each hidden layer neuron, *squash* the total net input using an *activation function* (here we use the *logistic function*), then repeat the process with the output layer neurons.

The Forward Pass

Here's how we calculate the total net input for h_1:

$$net_{h1} = w_1 * i_1 + w_2 * i_2 + b_1 * 1$$

$$net_{h1} = 0.15 * 0.05 + 0.2 * 0.1 + 0.35 * 1 = 0.3775$$

We then squash it using the logistic function to get the output of h_1:

$$out_{h1} = \frac{1}{1+e^{-net_{h1}}} = \frac{1}{1+e^{-0.3775}} = 0.593269992$$

Carrying out the same process for h_2 we get:

$$out_{h2} = 0.596884378$$

The Forward Pass

We repeat this process for the output layer neurons, using the output from the hidden layer neurons as inputs.

Here's the output for o_1:

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$net_{o1} = 0.4 * 0.593269992 + 0.45 * 0.596884378 + 0.6 * 1 = 1.105905967$$

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}} = \frac{1}{1+e^{-1.105905967}} = 0.75136507$$

And carrying out the same process for o_2 we get:

$$out_{o2} = 0.772928465$$

The Forward Pass

Calculating the Total Error

We can now calculate the error for each output neuron using the **squared error function** and sum them to get the total error:

$$E_{total} = \sum \frac{1}{2}(target - output)^2$$

For example, the target output for o_1 is 0.01 but the neural network output 0.75136507, therefore its error is:

$$E_{o1} = \frac{1}{2}(target_{o1} - out_{o1})^2 = \frac{1}{2}(0.01 - 0.75136507)^2 = 0.274811083$$

The Forward Pass

Repeating this process for ^{O_2} (remembering that the target is 0.99) we get:

$$E_{o2} = 0.023560026$$

The total error for the neural network is the sum of these errors:

$$E_{total} = E_{o1} + E_{o2} = 0.274811083 + 0.023560026 = 0.298371109$$